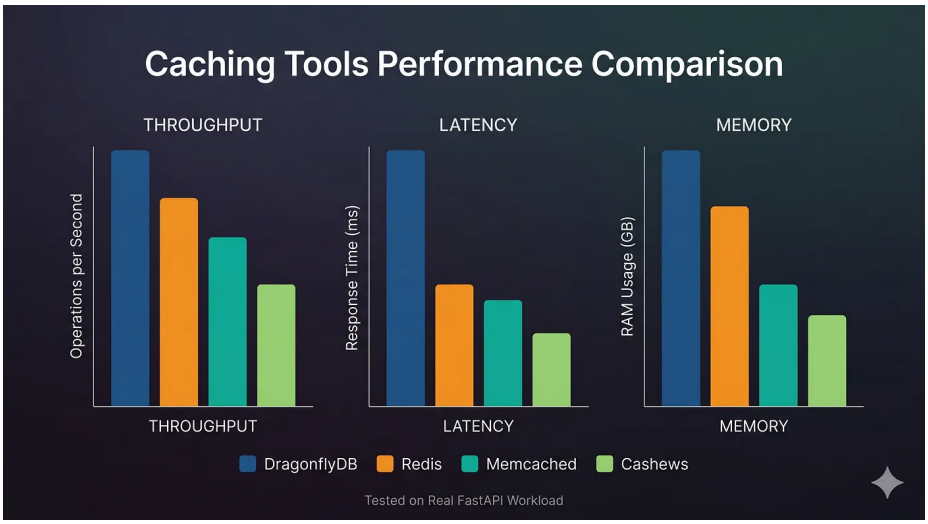
Stackademic

Member-only story

Redis Is Not the Best Python Cache Anymore. Here Is What Is.

I used Redis for three years without questioning it. One comparison changed that.

inprogrammer Follow 6 min read · Apr 19, 2026



Gemini (nano banana)

Redis is the default answer for Python caching. API responses, database speed, sessions, rate limiting. Everything points to Redis.

I followed the same path for three years without questioning it.

Then I tested it properly.

Four caching tools. Same FastAPI app. Same workload. Same hardware. Real production scenarios.

Redis did not come out on top.

Here is what actually performed better and why

it matters for your stack.

Friendlink for nonmembers-
https://medium.com/@inprogrammer/redis-is-not-the-best-python-cache-anymore-here-is-what-is-a4550b71b6e3?sk=d3a22974c006263bba52322cc66c28d6

The Test Setup

The application was a FastAPI service with three caching needs. Response caching for expensive API endpoints, session storage for authenticated users, and rate limiting on public endpoints.

I tested each tool against all three scenarios and tracked four metrics. Throughput under 500 concurrent requests, average latency per cache operation, memory usage per 100,000 cached items, and time to get something working from scratch.

Redis

Redis is the most widely deployed in-memory data store in the world. It supports strings, hashes, lists, sets, sorted sets, and more. It has

been production-hardened for over fifteen years.

Setup:

```
import redis.asyncio as redis
from fastapi import FastAPI
```

```
app = FastAPI()
cache =
redis.from_url("redis://localhost",
decode_responses=True)

@app.get("/products/{product_id}")
async def get_product(product_id: int):
    cached = await cache.get(f"product:
{product_id}")
    if cached:
        return json.loads(cached)
    product = await
fetch_from_db(product_id)
    await cache.setex(f"product:
{product_id}", 300, json.dumps(product))
    return product
```

Performance under 500 concurrent requests:

Average latency: 0.8ms per operation

Throughput: 142,000 operations per second

Memory per 100K items: 48MB

What it does well:

Redis handled every scenario correctly. Response caching, session storage, rate limiting all worked reliably. The ecosystem of Python libraries is mature. Every edge case is documented somewhere.

Where it fell short:

Redis requires a separate server process. For a small application or a development environment, running and maintaining a Redis instance adds operational overhead. Connection pooling requires careful configuration. Under very high concurrency I hit connection limit issues that required tuning.

Setup time to working code: 15 minutes.

Verdict: Reliable, fast, and battle-tested. The operational overhead is the only real cost.

Memcached

Memcached is the older alternative to Redis. Simpler data model, simpler operation, and a reputation for raw speed on simple key-value operations.

Setup:

```
from aiomcache import Client
```

```
cache = Client("127.0.0.1", 11211)

@app.get("/products/{product_id}")
async def get_product(product_id: int):
    key = f"product:{product_id}".encode()
    cached = await cache.get(key)
    if cached:
        return json.loads(cached)
    product = await
fetch_from_db(product_id)
    await cache.set(key,
json.dumps(product).encode(), exptime=300)
    return product
```

Performance under 500 concurrent requests:

Average latency: 0.6ms per operation

Throughput: 168,000 operations per second

Memory per 100K items: 31MB

What it does well:

Memcached is faster than Redis on pure key-value operations and uses less memory. If your caching needs are simple and you want maximum throughput on basic get and set

operations, Memcached wins on raw numbers.

Where it fell short:

No native support for complex data structures. Session storage required serializing everything to strings. Rate limiting required implementing the logic myself because Memcached has no atomic increment with expiry. No persistence means a restart clears everything. These limitations made it unsuitable for two of my three test scenarios.

Setup time to working code: 20 minutes including working around limitations.

Verdict: Faster than Redis on simple cases. Too limited for production applications with varied caching needs.

DragonflyDB

DragonflyDB is a Redis-compatible in-memory store designed as a modern Redis replacement. It is fully compatible with Redis clients and commands but built on a different architecture that claims 25x better memory efficiency and

higher throughput.

I included it expecting marketing claims that would not hold up. I was wrong.

Setup:

```
import redis.asyncio as redis
```

```
cache =
redis.from_url("redis://localhost:6379",
decode_responses=True)
```

That is the entire setup change. DragonflyDB speaks the Redis protocol so every existing Redis client works without modification. Replacing Redis with DragonflyDB in an existing application is changing one line in your configuration.

Performance under 500 concurrent requests:
Average latency: 0.5ms per operation
Throughput: 198,000 operations per second
Memory per 100K items: 19MB

What it does well:

DragonflyDB outperformed Redis on every metric. 39% lower latency, 39% higher throughput, 60% lower memory usage. All three test scenarios worked correctly because it supports the full Redis command set.

The memory efficiency difference is the most significant in production. Redis using 48MB per 100,000 items versus DragonflyDB using 19MB means you can cache significantly more data on the same hardware.

Where it fell short:

DragonflyDB is newer than Redis. The ecosystem of monitoring tools, debugging guides, and production war stories is smaller. I hit one edge case in cluster mode that had sparse documentation and required direct support.

Setup time to working code: 12 minutes. Faster than Redis because the client is identical.

Verdict: Outperforms Redis on every measurable metric with zero client code changes. The maturity gap is the only reason not to switch immediately.

Cashews

Cashews is a Python caching library built specifically for async Python applications. It wraps any backend including Redis, in-memory, and disk with a decorator-based API that eliminates most of the boilerplate.

Setup:

```
from cashews import cache
```

```
cache.setup("redis://localhost")

@app.get("/products/{product_id}")
@cache(ttl="5m", key="product:
{product_id}")
async def get_product(product_id: int):
    return await fetch_from_db(product_id)
```

The entire caching logic for an endpoint is one decorator. No manual get, set, or serialization.

No key construction. No TTL management. The decorator handles everything.

Performance under 500 concurrent requests:
Average latency: 0.9ms per operation
Throughput: 128,000 operations per second
Memory per 100K items: Depends on backend

What it does well:

Cashews eliminated 80% of the caching boilerplate in my codebase. Endpoints that previously required 8 lines of manual cache management became 1 line. The decorator pattern is clean and the cache invalidation API is the most developer friendly of the four tools tested.

Cache invalidation by pattern is particularly useful:

```
await cache.delete_match("product:*")
```

One line to invalidate all cached products. With

raw Redis this requires a SCAN operation and multiple DEL calls.

Where it fell short:

Cashews is an abstraction layer, not a cache itself. It still requires Redis or another backend underneath. Performance is slightly lower than direct Redis because of the abstraction overhead. For applications where raw cache throughput is the bottleneck, the overhead matters.

Setup time to working code: 8 minutes. Fastest of the four.

Verdict: The best developer experience of the four tools. Use it on top of DragonflyDB for both performance and ergonomics.

The Results

Raw performance numbers:

Throughput: DragonflyDB 198K ops/sec, Memcached 168K, Redis 142K, Cashews 128K

Latency: DragonflyDB 0.5ms, Memcached 0.6ms, Redis 0.8ms, Cashews 0.9ms

Memory per 100K items: DragonflyDB 19MB, Memcached 31MB, Redis 48MB, Cashews depends on backend

Developer experience: Cashews best, Redis good, DragonflyDB good, Memcached worst

Production maturity: Redis best, Memcached good, DragonflyDB growing, Cashews new

Why Redis Did Not Win

Redis is not slow. It is not bad. It lost this comparison because DragonflyDB is objectively faster and more memory efficient while being fully compatible, and Cashews provides a dramatically better developer experience for async Python applications.

The answer to “why not just use Redis” is not that Redis is wrong. It is that the tools that have built on Redis and around Redis have surpassed it in specific dimensions that matter for modern Python applications.

What I Use Now

For new projects I use DragonflyDB as the backend with Cashews as the Python interface. DragonflyDB handles the performance and memory efficiency. Cashews handles the developer experience. The combination gives better performance than Redis with better ergonomics than raw Redis client code.

For existing projects already running Redis, switching to DragonflyDB is genuinely worth evaluating. The client code does not change. The performance improvement is measurable. The main cost is the operational unfamiliarity with a newer tool.

CTA

If this changed how you think about caching in your Python applications, follow me here on Medium. I write about real Python and FastAPI production problems with exact fixes and actual numbers, not theory.

Running a different caching setup that has worked better for you in production? Drop it in

the comments. I read every one.

If you are setting up caching as part of a complete FastAPI production stack, this guide covers everything else you need: [Best Python FastAPI Production Tools 2026]

Python Fastapi Software Development Web Development Programming

Responses (2)

Pancake Agree

What are your thoughts?

See all responses